07. Writing Large Programs

Kyu-haeng Lee

SW Convergence Coding II

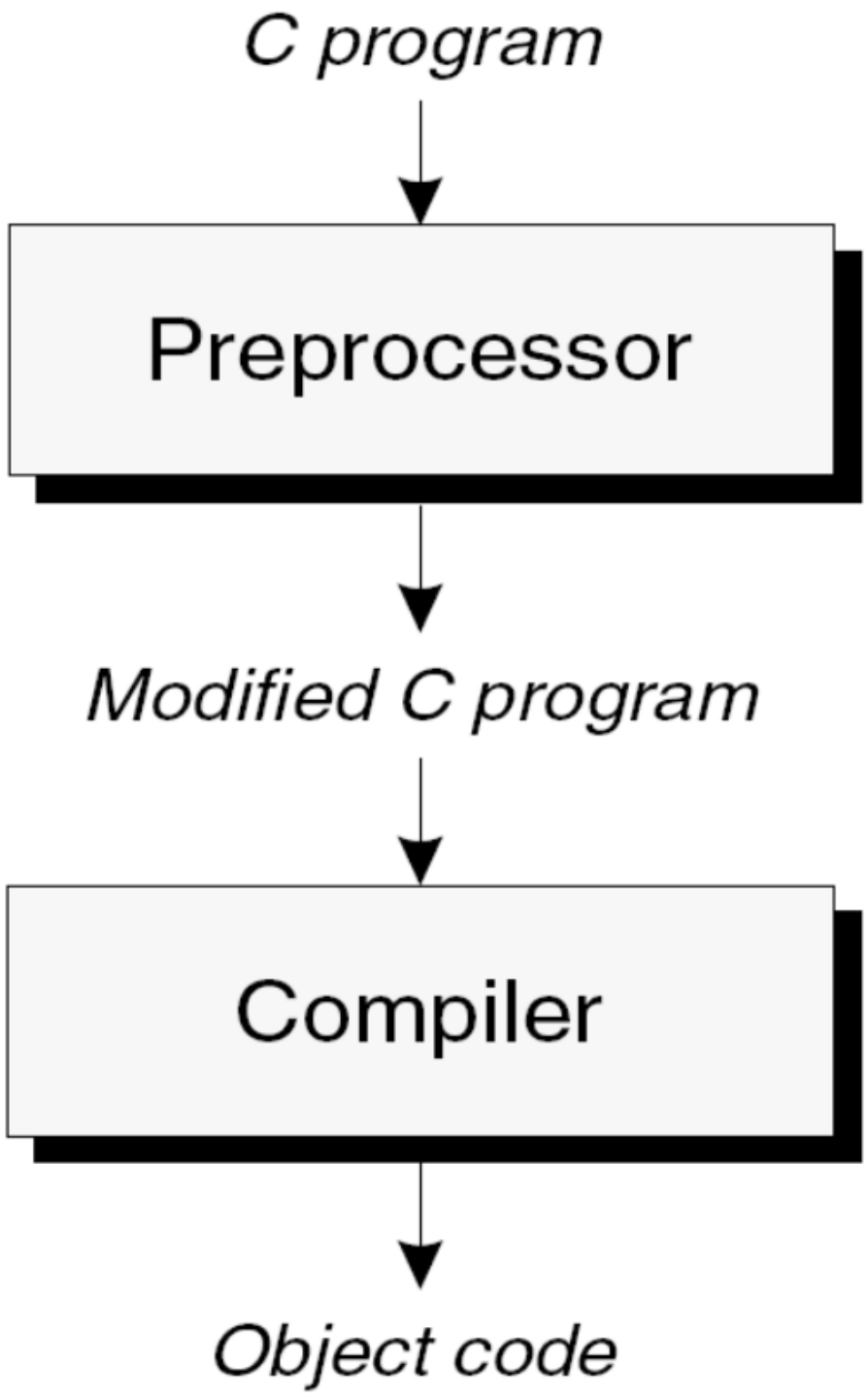
Fall 2025



Preprocessor

The Preprocessor

- Directives such as
  #define and #include
  are handled by the preprocessor,
  a piece of software
  that edits C programs
  just prior to compilation.



The Preprocessor

- Preprocessor directives begin with a #
  - File inclusion
    - #include – includes a named file
  - Macro definition
    - #define – defines a (text replacement) macro
  - Conditional compilation
    - #ifdef/#else/#endif – conditional compilation

```
#ifdef MACRONAME
//part 1
#else
//part 2
#endif
```

Preprocessor Directives

- Several rules apply to all directives.
  - Directives always begin with the # symbol.
  - Directives can appear anywhere in a program.

- Directives always end at the first new-line character, unless explicitly continued.
  - To continue a directive to the next line, end the current line with a \ character:

```
#define DISK_CAPACITY ( SIDES * \
                        TRACKS_PER_SIDE * \
                        SECTORS_PER_TRACK * \
                        BYTES_PER_SECTOR)
```

#define

- Often used to define constants
  - #define TRUE 1
  - #define FALSE 0
  - #define PI 3.14159
  - #define SIZE 20
- Offers easy one-touch change of scale/size

- #define vs constants
  - The preprocessor directive uses no memory

#define - more readable

```c
#include<stdio.h>
#define MILE 1
#define KM 2
void km_mile_conv(int choice) {
    // …
    if (choice == MILE)
    // …
}
int main() {
    // …
    switch (choice) {
    case MILE:
        km_mile_conv(choice);
        break;
    case KM :
        km_mile_conv(choice);
        break;
    /* more cases */
    }
}
```

Parameterized Macros

- Examples of parameterized macros:
  - #define MAX(x,y) ((x)>(y)?(x):(y))
  - #define IS_EVEN(n) ((n)%2==0)
- Invocations of these macros:
  - i = MAX(j+k, m-n);
  - if (IS_EVEN(i)) i++;
- The same lines after macro replacement:
  - i = ((j+k)>(m-n)?(j+k):(m-n));
  - if (((i)%2==0)) i++;

Parameterized Macros

- A more complicated function-like macro:
  - #define TOUPPER(c) \
  - ('a'<=(c)&&(c)<='z'?(c)-'a'+'A':(c))

Parameterized Macros

- A macro may evaluate its arguments more than once.
  - Unexpected behavior may occur if an argument has side effects:
  - #define MAX(x,y) ((x)>(y)?(x):(y))
  - n = MAX(i++, j);

  - The same line after preprocessing:
  - n = ((i++)>(j)?(i++):(j));

  - Errors caused by evaluating a macro argument more than once can be difficult to find, because a macro invocation looks the same as a function call.
  - For self-protection, it's a good idea to avoid side effects in arguments.

The # Operator

- The # operator converts a macro argument into a string literal; it can appear only in the replacement list of a parameterized macro.
- Example
  - #define PRINT_INT(n) printf(#n " = %d\n", n)
- The invocation
  - PRINT_INT(i/j); will become
  - printf("i/j" " = %d\n", i/j);
  - printf("i/j = %d\n", i/j); //same to above
    - The compiler automatically joins adjacent string literals.

The ## Operator

- The ## operator can “paste” two tokens together to form a single token.
  - If one of the operands is a macro parameter, pasting occurs after the parameter has been replaced by the corresponding argument.

The ## Operator

- Example
  - #define MK_ID(n) i##n

  - A declaration that invokes MK_ID three times:
  - int MK_ID(1), MK_ID(2), MK_ID(3);

  - The declaration after preprocessing:
  - int i1, i2, i3;

General Properties of Macros

- Macros may be “undefined” by the #undef directive.
  - The #undef directive has the form

    #undef identifier
  - where identifier is a macro name.

  - One use of #undef is to remove the existing definition of a macro so that it can be given a new definition.

Parentheses in Macro Definitions

- The replacement lists in macro definitions often require parentheses in order to avoid unexpected results.
- Each occurrence of a parameter in a macro's replacement list needs parentheses as well:
  - #define SCALE(x) (x*10) /* needs parentheses around x */
- During preprocessing, the statement
  - j = SCALE(i+1);

    becomes
  - j = (i+1*10);
  - j = i+10; //Equivalent to the above.

Conditional Compiling

- Debugging (so that you don't have to remove all your printf debugging!)
  - #ifdef DEBUG
    - // lots and lots of printfs
  - #else
    - // nothing often omitted
  - #endif
- Portability
  - #ifdef WINDOWS
    - // code that only works on windows
  - #endif

Defining a Macro for #ifdef

- #define DEBUG
- #define DEBUG 0
- #define DEBUG 1

- The –Dmacro[=def] flag of gcc
  - gcc –DDEBUG hw1.c –o hw1
  - gcc –DDEBUG=1 hw1.c –o hw1
  - gcc –DDEBUG=0 hw1.c –o hw1

#ifndef, #if, #elif, #else

- #ifndef is the opposite of #ifdef
- #if DEBUG number(int)
  - Test to see if DEBUG is an integer number
    - ex) #define DEBUG 1
    - Undefined macros are considered to be 0.
- #elif MACRONAME
  - #if WINDOWS
    - //included if WINDOWS is non-zero
  - #elif LINUX
    - //included if WINDOWS is zero, but LINUX is non-zero
  - #else
    - //if both are 0
  - #endif

Predefined Macros

- Useful macros that primarily provide information about the current compilation
  - __LINE__ Line number of file compiled
  - __FILE__ Name of file being compiled
  - __DATE__ Date of compilation
  - __TIME__ Time of compilation

- printf("Comipiled on %s at %s.\n", __DATE__, __TIME__);

#error

- #error message
  - prints message to screen
  - often used in conjunction with #ifdef, #else

  - #if WINDOWS
    - //…
  - #elif LINUX
    - //…
  - #else
    - #error OS not specified
  - #endif

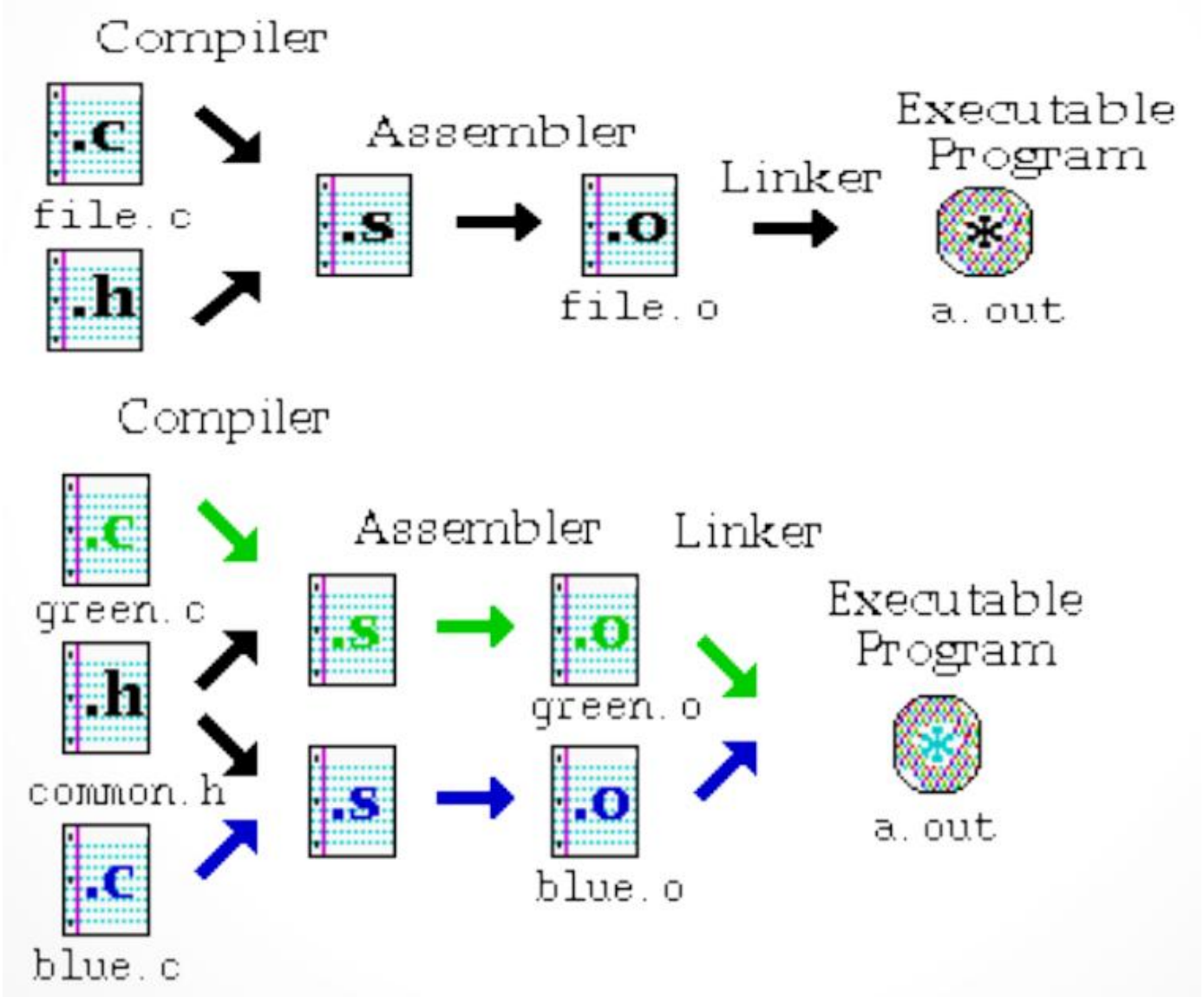
Writing a large program

The Build Process

- Preprocessing:
  - processes include-files, conditional compilation and macros.
- Compilation:
  - takes the output of the preprocessor and the source code, and generates assembler source code, i.e., making .s file.
- Assembly:
  - takes the assembly source code and produces an assembly listing with offsets. The assembler output is stored in an object file, i.e., .o file.
- Linking:
  - takes one or more object files or libraries as input and combines them to produce a single (usually executable) file.

The Build Process

gcc case



Multiple Source Files

- How to divide your program?

- Some guidelines
  - One and only one .c file must contain a main.
  - Functions that handle some common aspects of a program should be grouped into the same file.
  - main, data structure implementation (i.e. linked list), I/O, utilities, display/GUI, etc.

Multiple Source Files

- Concerns
  - How to access functions/variables/macros in other files?

- Make headers
  - The #include tells the preprocessor to insert the contents of a specified file.

Header Files

- Header File
  - Contains a collection of function prototypes, constant and preprocessor definitions
  - A way to use functions defined in other source files
  - Named with extension .h
    - By convention a header file carries the same name as the associated .c file
    - ex) hw1.h → hw1.c

- Included in the source file with #include
  - #include <stdio.h>
  - #include "hw1.h"

Header Files

- Two primary forms of #include
  - The difference between the two has to do with how the compiler locates the header file.

- For C's own library: #include <filename>
  - Search the directories in which system header files reside.
- For all other header files: #include "filename"
  - Search the current directory, then search the directories in which system header files reside.

Header Files

- Don't use brackets when including header files that you have written:
  - #include <myheader.h> /*** WRONG ***/
  - The preprocessor will probably look for myheader.h where the system header files are kept.

Header Files

- Sharing Function Prototypes
  - If a function is to be called in more than one file, put its prototype into a .h.
  - Always include the .h with f's prototype in the .c that contains f's definition.
  - A header file should never contain function definitions.

```c
//add.h

int add(int a, int b);
```

```c
//add.c

#include "add.h"

int add(int a, int b) { return a + b; }
```

```c
//main.c

#include <stdio.h>
#include "add.h"
int main()
{
    printf("%d", add(1, 2));
    return 0;
}
```

Header Files

- The file name in an #include directive may include information that helps locate the file, such as a directory path or drive specifier:
  - #include "c:\cprogs\utils.h " /* Windows path */
  - #include "/cprogs/utils.h " /* UNIX path */

- Although the quotation marks in the #include directive make file names look like string literals, the preprocessor doesn't treat them that way.

Header Files

- It's usually best not to include path or drive information in #include directives.
- Bad examples of Windows #include directives:
  - #include "d:utils.h"
  - #include "\cprogs\include\utils.h"
  - #include "d:\cprogs\include\utils.h"
- Better versions:
  - #include "utils.h"
  - #include "..\include\utils.h"

Sharing Variables

- extern
  - extern int i, a[];
  - extern informs the compiler that the variables i and a are defined elsewhere.
  - extern declarations often go in to a header file.

Sharing Variables

- The variable must have only one definition among all files.
  - int x;
- Any file that wishes to access a variable that is defined in another file must declare such a variable as extern
  - extern int x;
- When we use extern in the declaration of an array, we can omit the length of the array:
  - extern int a[];
  - No need for it to know a's length.

Sharing Variables

- c is defined as extern.

```c
//add.h

int add(int a, int b);
```

```c
//add.c

#include "add.h"
int c=3;
int add(int a, int b) { return a + b + c; }
```

```c
//main.c

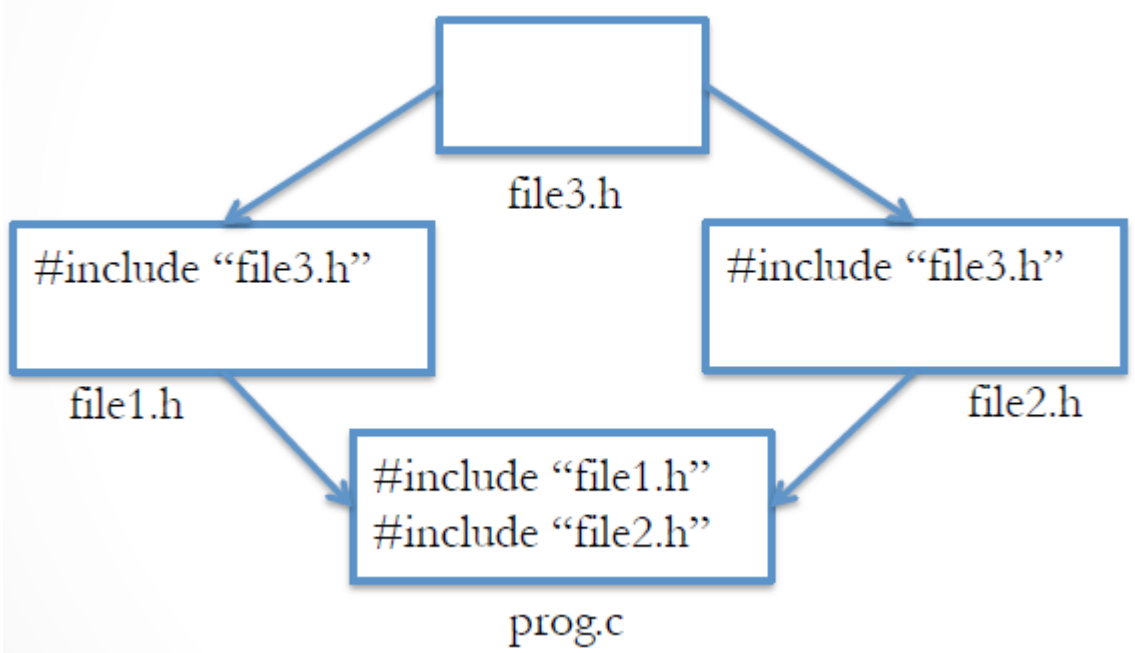
#include <stdio.h>
#include "add.h"
extern int c;

int main()
{
    printf("%d %d", add(1, 2), c);

    return 0;
}
```

Protecting Header Files

- If a source file includes the same header file twice, compilation errors may result.



- When prog.c is compiled, file3.h will be compiled twice.

Protecting Header Files

- Including the same header file twice doesn't always cause a compilation error.
- If the file contains only macro definitions, function prototypes, and/or variable declarations, there won't be any difficulty.
- If the file contains a type definition, however, we'll get a compilation error.

Protecting Header Files

- To protect a header file, we'll enclose the contents of the file in an #ifndef-#endif pair.

- How to protect the boolean.h file:
  - #ifndef BOOLEAN_H
  - #define BOOLEAN_H
  - #define TRUE 1
  - #define FALSE 0
  - typedef int Bool;
  - #endif

```
//add.h

#ifndef ADD_H
#define ADD_H

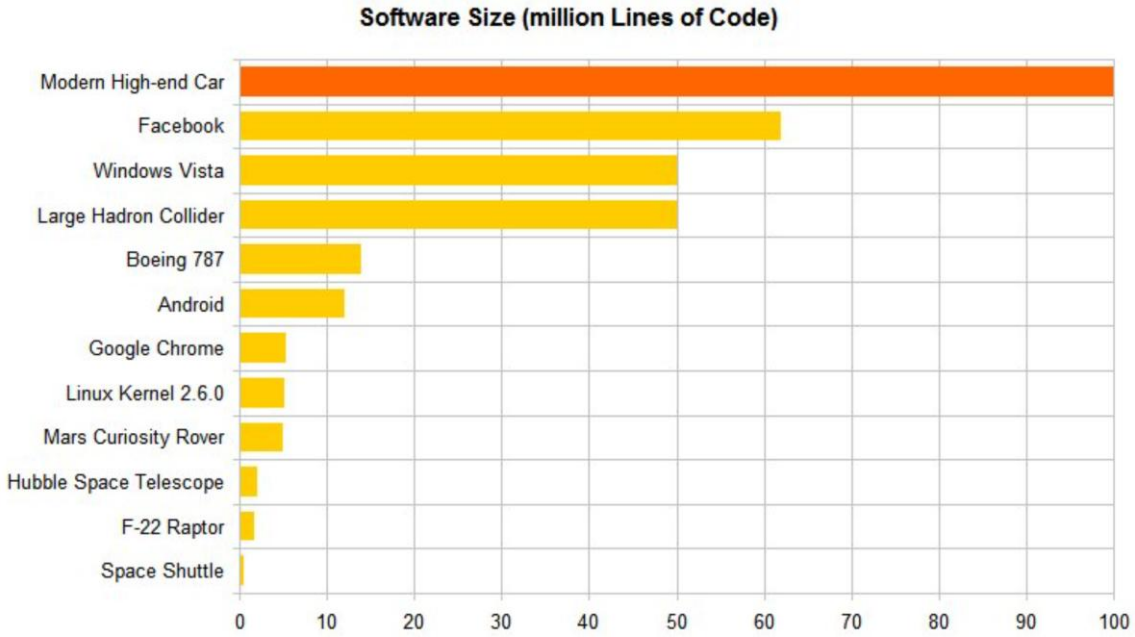
int add(int a, int b);

#endif
```

Programming-in-the-large

- Most full-featured programs today are at least 1 million lines long.

- Careful design
  - Do not limit any possible future extensions
- Extreme attention to style
  - Lots of people are going to be working on it
- Planning for maintenance
  - It is going to be modified many times

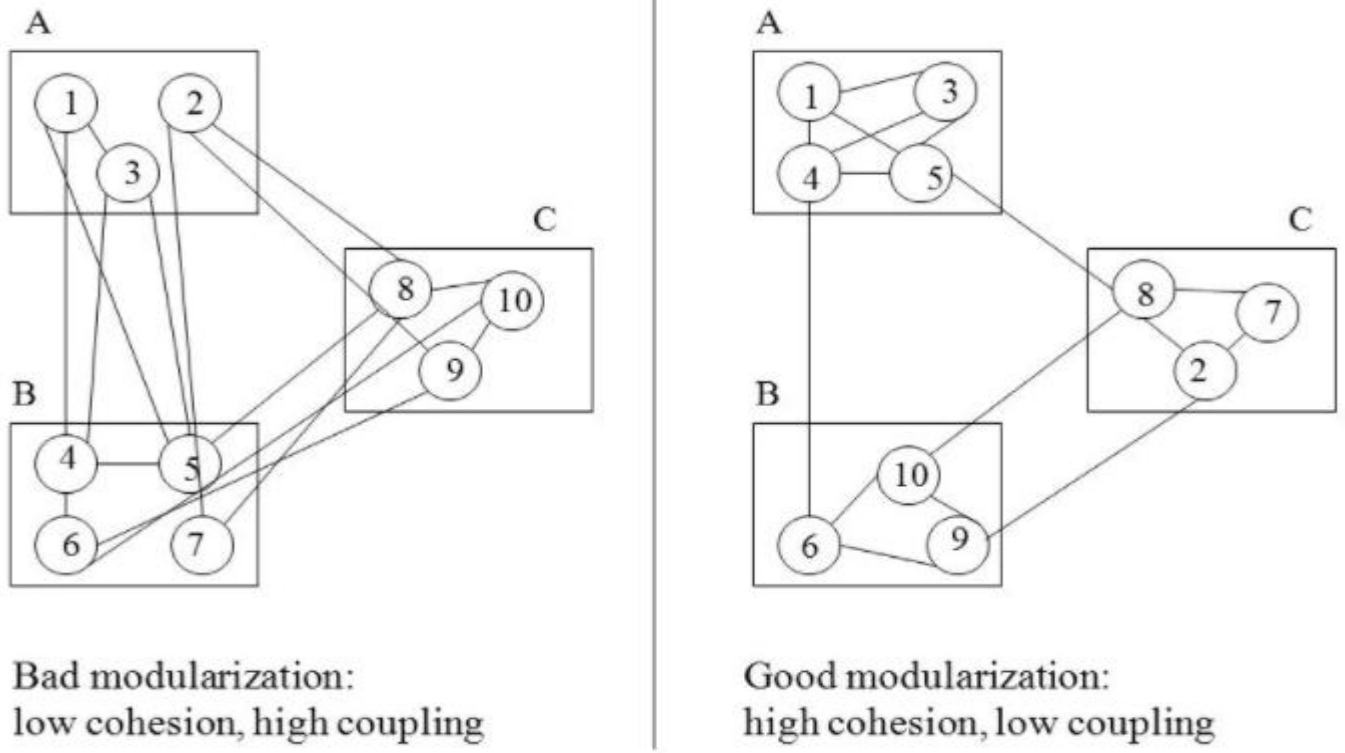


Module-based Programming

- A module is a collection of services (functions), some of which are available to other parts of the program (clients) through an interface (header file).
  - Abstraction – carefully designed modules should hide implementations
  - Reusability – in order to maximize this feature, modules should be as basic and as minimal as possible
  - Maintainability – easier to locate and fix bugs

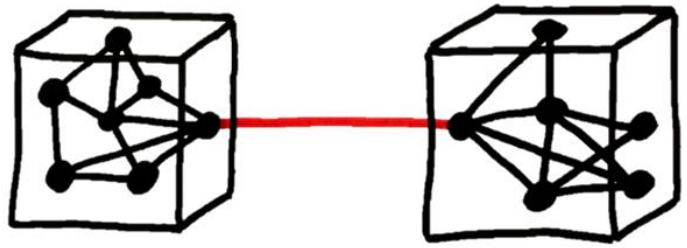
Cohesion and Coupling

- High cohesion
  - Elements in each module should be closely related
- Low coupling
  - Modules should be as independent of each other as possible



Bad modularization:
low cohesion, high coupling

Good modularization:
high cohesion, low coupling



Low Cohesive

MyReader
validateLocation() checkFTP() ping() readFromDisk() readFromWeb() readFromNetwork()

High Cohesive

MyReader
readFromDisk() readFromWeb() readFromNetwork()

https://www.javaguides.net/2018/08/cohesion-in-java-with-example.html

https://www.planetgeek.ch/2011/07/08/presentation-agile-code-design-how-to-keep-your-code-flexible/

References

- Chapter 13. C Preprocessor, C: How to Program, Paul Deitel et al.
- Chapter 19. Program Design, C Programming: A Modern Approach, K. N. KING.